CAWASMA: response to the Mid-2 review

Context-Aware Web and API Security Misconfiguration Analyzer · Final-year project · November 2026

This document answers the four points raised in the review of the Mid-2 slides: how CAWASMA relates to existing scanners, how its output compares with theirs, what the working system looks like, and what is planned for the areas it does not cover.

Main points

- **Position.** In the documentation we read for ZAP, Nuclei, Nikto, Burp Suite Professional and Acunetix/Invicti, we found no automatic re-scoring of an individual finding from the endpoint it is on and the data that endpoint returned. CAWASMA does not replace these tools; it does not test for injection, XSS or broken object-level authorisation.
- **Detection.** CAWASMA's mean recall is 0.76, close to Nuclei's 0.80 and above the 0.44 of ZAP's passive baseline, on four intentionally vulnerable targets. The comparison favours CAWASMA's scope, as Section 2 explains.
- **Ranking.** On four targets, the context-aware ranking is better than the base score only on DVWA and is otherwise equal or slightly worse, so this evaluation does not show that it ranks better overall.
- **Next step.** One item is proposed before final submission: a safe XSS marker-reflection check (2–3 days).

Tool	Findings (mean)	TP / FP / FN (total)	Not covered (total)	Precision (mean)	Recall (mean)	FPR (mean)	Scan time (mean, s)
CAWASMA	307.5	25 / 0 / 8	0	1.00	0.76	0.00	10.5
ZAP (baseline)	23.3	7 / 0 / 14	40	1.00	0.44	0.00	104.5
Nuclei	13.5	13 / 0 / 4	41	1.00	0.80	0.00	126.3

Limits of this work

- Only ZAP and Nuclei were run. Nikto, Burp and Acunetix/Invicti are described from their documentation.
- TLS and CORS have no ground-truth entries, so they could not be scored.
- The graded priority pools are not finished, so there is no pool-based NDCG yet.
- The scan shown in Section 3 produced no attack chains, so no chain screenshot is included. In the evaluation, chains appear only on DVWA and crAPI.

1. Existing scanners

These tools are good at finding vulnerabilities, and several find classes CAWASMA does not look for at all (see Section 4). The gap CAWASMA addresses is narrower: **scoring the same finding differently according to the endpoint it is on and the data that endpoint returned**.

Everything below comes from each tool's own documentation, read on 2026-11-09. "Not described" means we found no such feature in the pages linked; it does not prove the feature is absent.

Scanner	Type	What it does well	How it rates severity	What it does not do that CAWASMA does
OWASP ZAP (ZAP by Checkmarx)	Open source, Apache-2.0 (site, repo)	Passive and active scan rules for specific vulnerabilities (rules); a passive baseline mode for CI/CD that makes no attacks (baseline)	Each scan rule has a set risk (High, Medium, Low, Informational), plus a confidence (Low, Medium, High). A user can override risk per alert or with an Alert Filter (alerts, filters)	Automatic re-scoring of a finding from endpoint sensitivity or returned data: not described. Overrides are configured by the user
Nuclei (ProjectDiscovery)	Open source, MIT (repo); a paid cloud service is built on it	Fast, customisable, template-based scanning; templates reproduce real steps to verify an issue and cut false positives (repo, templates)	Each template declares its own `severity` (structure): info, low, medium, high, critical or unknown (CLI help)	Changing a template's severity from the endpoint or the response: not described
Nikto	Open source, GPL-3.0; its database files are under separate terms (repo)	Web server scanning, with output in CSV, HTML, XML, JSON and SQL (export formats)	No severity rating: its report fields are id, scan id, test id, host details, references, method, URI, message, request and response (export formats)	Any severity or ranking of findings
Burp Suite Professional (PortSwigger)	Commercial, $499 listed (product)	Automated scanning with manual penetration-testing tools in one product (product)	Severity (High, Medium, Low, Information) reflecting "the likely impact of each issue for a typical organization", plus confidence (Certain, Firm, Tentative) (sample report). Each issue type has a typical severity (definitions). A user can reassign both (issues)	Automatic re-scoring from endpoint sensitivity or returned data: not described. Its advisories discuss context in prose, for a person to judge
Acunetix / Invicti ("Acunetix is now Invicti Web + API")	Commercial (pricing)	Proof-based scanning: confirms vulnerabilities "by attempting safe, read-only exploits" (proof-based)	Levels by vulnerability type: Critical, High, Medium, Low, plus Best Practice and Information (levels); supports CVSS v3.0 (FAQ)	Per-finding re-scoring by endpoint context: not described. Predictive Risk Scoring and Business Criticality work at website or target level (below)

The closest commercial idea: Invicti Predictive Risk Scoring

Announced by Invicti on 22 April 2024 (blog). It is the nearest thing we found to our idea, and it solves a different problem:

	Invicti Predictive Risk Scoring	CAWASMA
What gets a score	Each discovered **website**	Each **finding on an endpoint**
When	**Before** scanning ("an early pre-scan step")	**After** a finding is detected
Inputs	Up to 220 public attributes of the site, such as deprecated TLS or the number of form inputs (docs)	The endpoint's sensitivity class and the data signals in its response
Output	A prediction of the highest-severity vulnerability on the site, so teams know which sites to scan first	A CVSS v3.1 environmental score, with a vector anyone can recompute

Invicti says it "is not a substitute for scanning". It decides which site to scan first; CAWASMA decides which finding to fix first. They are complementary. The vendor also reports a confidence of at least 83 %, which we have not checked.

Acunetix also lets a user set a per-target **Business Criticality** (Low, Normal, High, Critical), used to "filter scans and vulnerabilities" (targets). It is set by a person, for a whole target; the page does not say it changes a finding's severity.

What this does and does not show

- CAWASMA does not replace these tools. It runs safe, read-only checks for misconfigurations and does not test for injection, XSS or broken object-level authorisation.
- We evaluated only ZAP and Nuclei (Section 2). Nikto, Burp and Acunetix/Invicti appear here from their documentation only: we did not run them.

2. Output comparison with ZAP and Nuclei

Data: the evaluation run `20261105T050013Z` (5 November 2026), on commit `5f2d60e`, all targets reset, all three tools run together. The evaluation was not repeated for this document: between `5f2d60e` and current `main` (`4051a16`) the scanner's code differs only in formatting and an unused `status` field on specs, so detection and scoring are unchanged.

Tools: CAWASMA 1.0.0 (depth 2, wordlist on, active probes); ZAP 2.15.0 baseline scan (VAmPI: API scan, safe mode); Nuclei 3.9.0 with templates v10.1.1.

Every table is also available as a CSV file in `docs/supervisor/` in the repository.

How to read these numbers fairly

- **ZAP ran in baseline mode**, which by design "doesn't perform any actual 'attacks'". These results don't represent a full active ZAP scan.
- **ZAP and Nuclei are scored only on checks their rules map to.** *Not covered* counts the ground-truth entries each could not be scored on. CAWASMA is scored on every entry, and the ground truth uses CAWASMA's check ids, so the benchmark favours CAWASMA's scope.
- **The endpoint rule is strict:** a missing header reported on `/` doesn't match an entry on `/login.php`, which lowers ZAP's and Nuclei's recall.
- **Finding counts aren't comparable.** CAWASMA reports each check once per endpoint; Nuclei reports mostly once per host. More findings doesn't mean better.
- **Precision covers labelled findings only.** Most findings match no ground-truth entry (CAWASMA: 1,184 of 1,230), so they are counted neither right nor wrong.
- The ground truth is a single rater's first pass. Four targets are too few for any significance claim.

a. Per target and per tool

Target	Tool	Findings	TP	FP	FN	Not covered	Precision	Recall	FPR	Scan time (s)
Juice Shop	CAWASMA	603	4	0	5	0	1.00	0.44	0.00	23.4
Juice Shop	ZAP	49	1	0	3	12	1.00	0.25	0.00	324.4
Juice Shop	Nuclei	11	3	0	2	11	1.00	0.60	0.00	198.8
DVWA	CAWASMA	115	10	0	0	0	1.00	1.00	0.00	4.3
DVWA	ZAP	12	0	0	9	7	undefined	0.00	0.00	36.5
DVWA	Nuclei	16	3	0	2	7	1.00	0.60	0.00	102.3
VAmPI	CAWASMA	68	6	0	2	0	1.00	0.75	0.00	6.8
VAmPI	ZAP	27	2	0	2	13	1.00	0.50	0.00	24.4
VAmPI	Nuclei	13	3	0	0	14	1.00	1.00	0.00	102.6
crAPI	CAWASMA	444	5	0	1	0	1.00	0.83	0.00	7.5
crAPI	ZAP	5	4	0	0	8	1.00	1.00	0.00	32.5
crAPI	Nuclei	14	4	0	0	9	1.00	1.00	0.00	101.3

Averaged across the four targets:

Tool	Findings (mean)	TP / FP / FN (total)	Not covered (total)	Precision (mean)	Recall (mean)	Recall (pooled)	FPR (mean)	Scan time (mean, s)
CAWASMA	307.5	25 / 0 / 8	0	1.00	0.76	0.76	0.00	10.5
ZAP	23.3	7 / 0 / 14	40	1.00*	0.44	0.33	0.00	104.5
Nuclei	13.5	13 / 0 / 4	41	1.00	0.80	0.76	0.00	126.3

* ZAP's mean precision is over three targets: on DVWA it had no true or false positives, so precision is undefined there.

CAWASMA and Nuclei have about the same recall, but Nuclei's is measured on 17 entries and CAWASMA's on 33. All three show zero false positives, because a false positive is counted only against an explicit `absent` or `must_not_fire` entry.

b. Coverage by check category

A cell counts `present` ground-truth entries matched. **detected** = all, **partly** = some, **not detected** = none.

Category	Checks in ground truth	CAWASMA	ZAP	Nuclei
Security headers	HDR-002..007	detected (15/15)	partly (7/15)	partly (12/15)
Cookies	CKI-001..003	detected (3/3)	not detected (0/3)	not detected (0/3; no rule mapped)
Information disclosure	INF-002, INF-003	partly (3/5)	not detected (0/5)	partly (1/5)
API auth / rate limit / data exposure	API-001, API-003	partly (4/10)	not detected (0/10; no rule mapped)	not detected (0/10; no rule mapped)
TLS	none	not measured	not measured	not measured
CORS	none	not measured	not measured	not measured

TLS and CORS have no `present` entries in the ground truth, so they cannot be scored. What the run did output for them:

- **TLS:** no findings from any tool. CAWASMA reaches the targets through our own nginx TLS front, so its TLS checks see that front rather than the targets; ZAP and Nuclei scanned the targets over plain HTTP.
- **CORS:** on Juice Shop, CAWASMA reported 166 findings (COR-001, COR-004, COR-005) and ZAP 11 ("Cross-Domain Misconfiguration"). Nuclei reported none. No other target produced a CORS finding.

c. Three side-by-side examples

In the 46 (check, endpoint) pairs that CAWASMA shares with ZAP or Nuclei, CAWASMA **raised 11, kept 34 and lowered 1**. The one lowered pair is INF-006 on VAmPI's `/openapi.json` (5.3 → 3.2), under the interim legacy formula.

	1. Raised: same issue, sensitive endpoint	2. Kept: same issue, public page	3. Raised by response data
Target, endpoint	VAmPI `/users/v1/_debug`	VAmPI `/`	crAPI `/.env`
Issue	HDR-004: missing X-Content-Type-Options	HDR-004: missing X-Content-Type-Options	INF-003: `.env` file exposed
ZAP risk (confidence)	Low (Medium)	Low (Medium)	no alert on this endpoint
Nuclei severity	no result on this endpoint	info	high
CAWASMA base score	4.3 (Medium)	4.3 (Medium)	9.1 (Critical)
CAWASMA final score	**5.0** (Medium)	**4.3** (Medium)	**9.8** (Critical)
Endpoint class	HIGH	LOW	LOW
Body signals	CREDENTIALS, PII	none	CREDENTIALS
Requirements	CR:H / IR:H / AR:M	CR:X / IR:X / AR:X	CR:H / IR:H / AR:M
Why CAWASMA moved it	sensitive endpoint returning credentials and personal data	public page, no sensitive data: the base score stands	the response holds credentials, which raises the score even though the path class is LOW

Examples 1 and 2 are the point of the project: ZAP gives the same rating on both endpoints, while CAWASMA separates them. The change can be modest: in example 1 the severity label stays Medium.

d. Ranking result, as it stands

NDCG@10 of each ranking against the hand-written priority list; mean over 1,000 random tie-breaks.

Target	Environmental-scored only: base → environmental	All findings mixed: base → context	Secondary, grouped by check: base → environmental
Juice Shop	0.001 → 0.000	0.000 → 0.000	0.633 → 0.635
DVWA	0.147 → 0.261	0.146 → 0.261	0.843 → 0.834
VAmPI	0.607 → 0.591	0.602 → 0.410	0.667 → 0.653
crAPI	0.539 → 0.535	0.538 → 0.535	0.821 → 0.808
Mean	**0.324 → 0.347**	**0.322 → 0.301**	**0.741 → 0.733**

The grouped view was added after the first results, as a diagnostic, and is not the headline. The graded pools (`eval/ground_truth/priority_pool_*.csv`) have no grades yet, so there is no pool-based NDCG.

What it means: on four targets, the context-aware ranking is better than the base score only on DVWA and is otherwise equal or slightly worse, so this evaluation does not show that it ranks better overall.

3. The system in use

Screenshots from a fresh scan of the VAmPI test target on 9 November 2026 (68 findings).

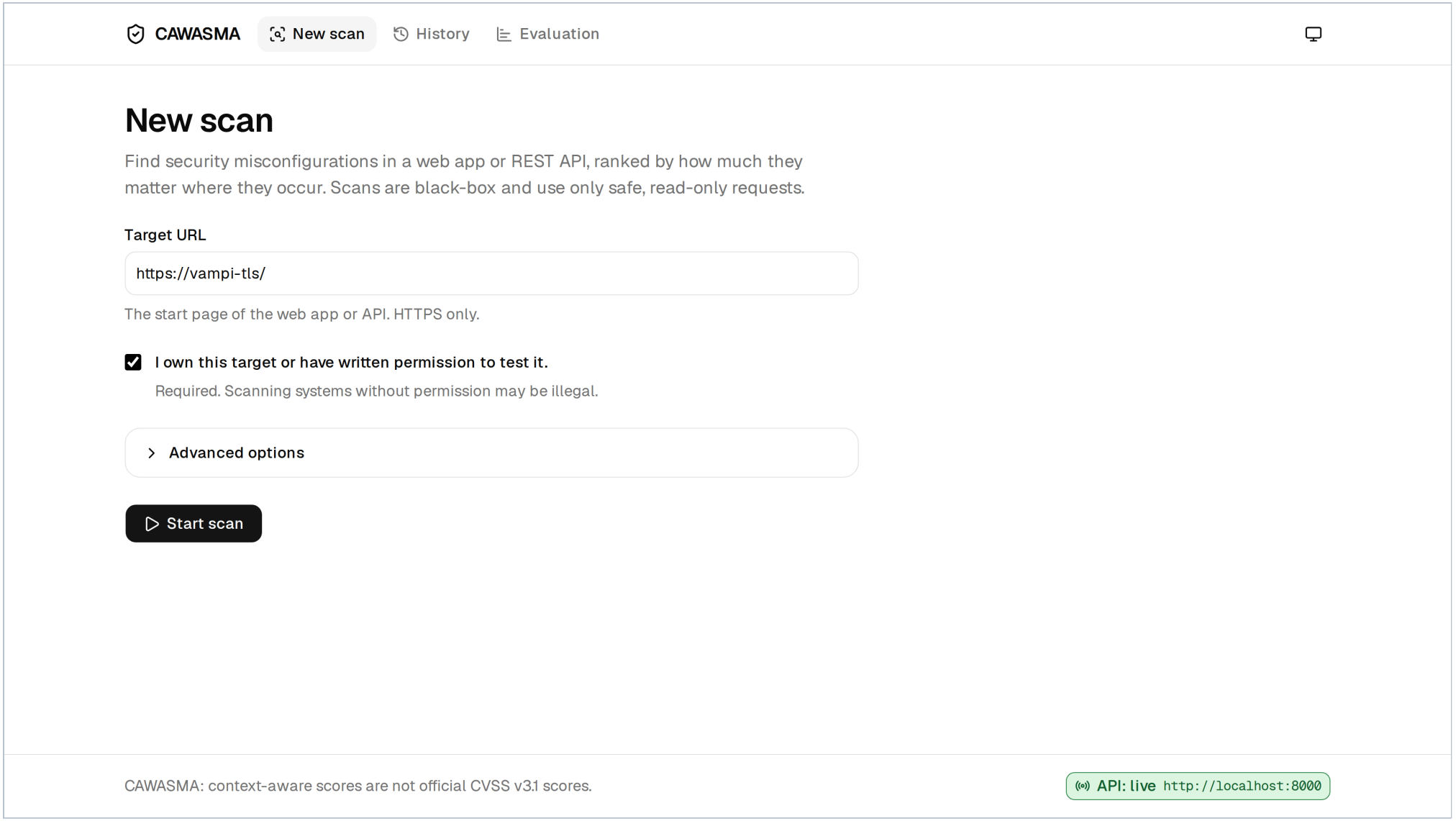


Figure 1. The new-scan form with the target filled in and the required permission confirmation ticked.

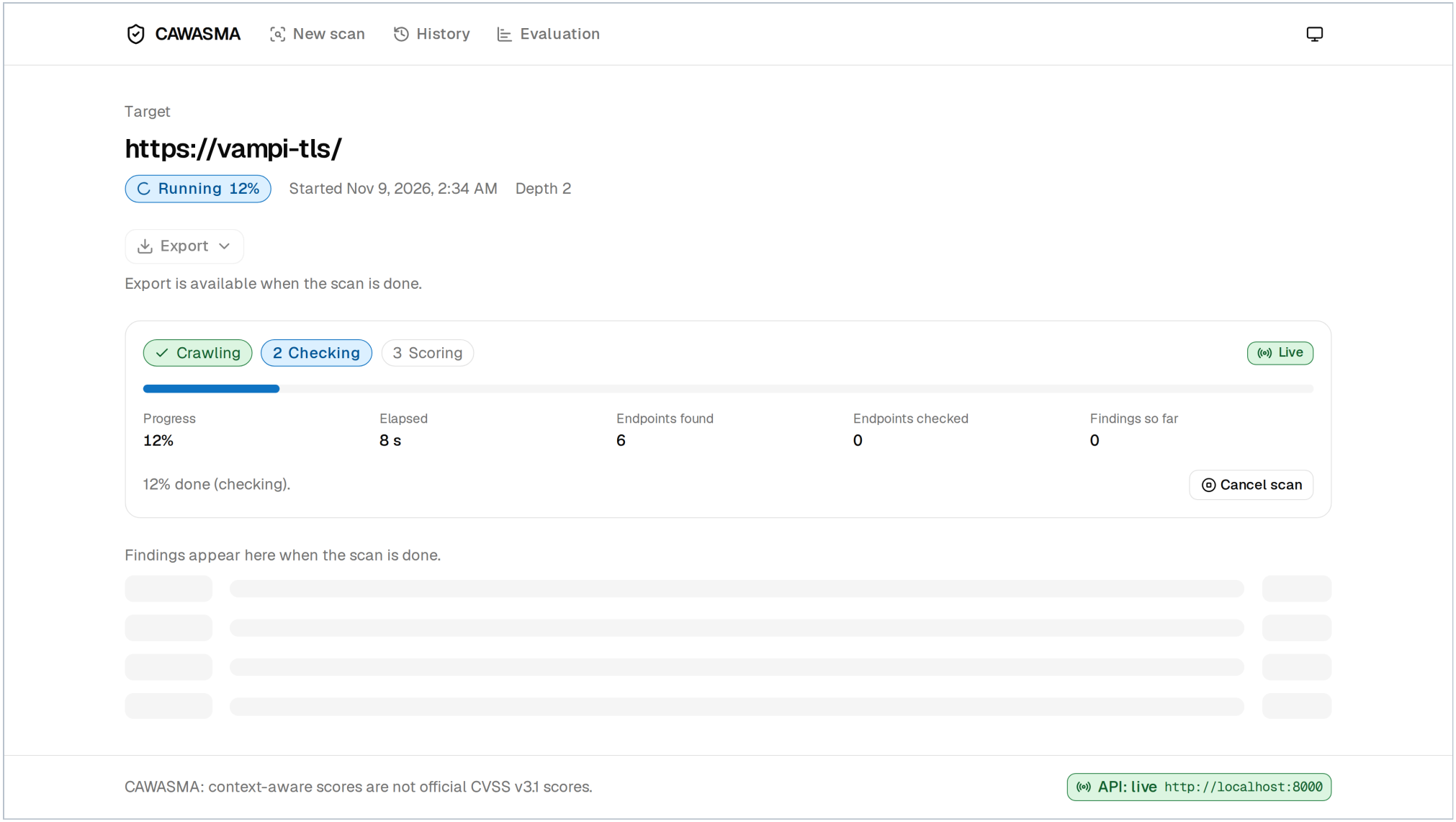


Figure 2. The scan in progress, 12 % done, with its stage, elapsed time and endpoints found updating live.

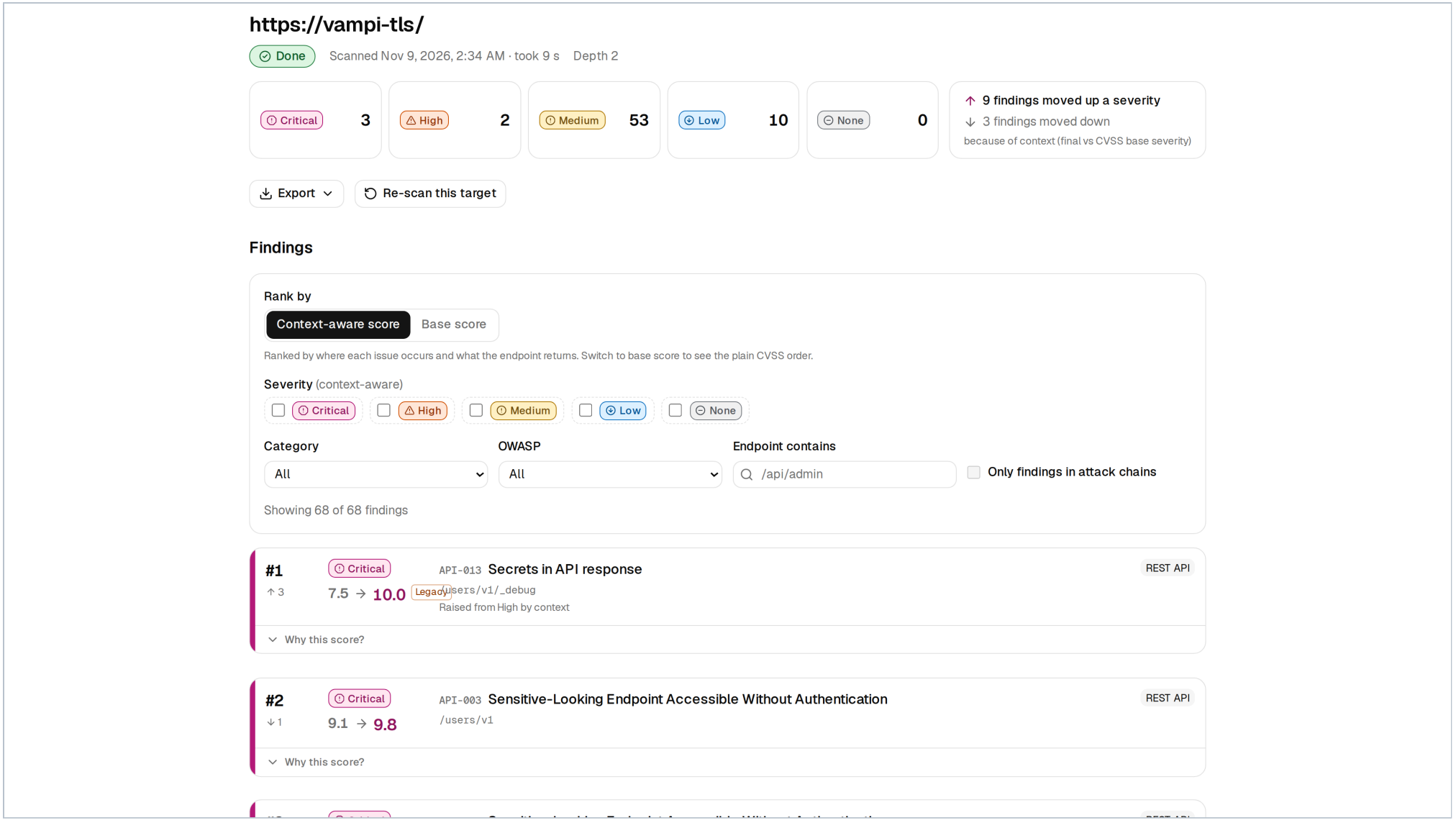


Figure 3. Results for VAmPI: severity summary (3 critical, 2 high, 53 medium, 10 low), 9 findings moved up and 3 down by context, and the list ranked by context-aware score.

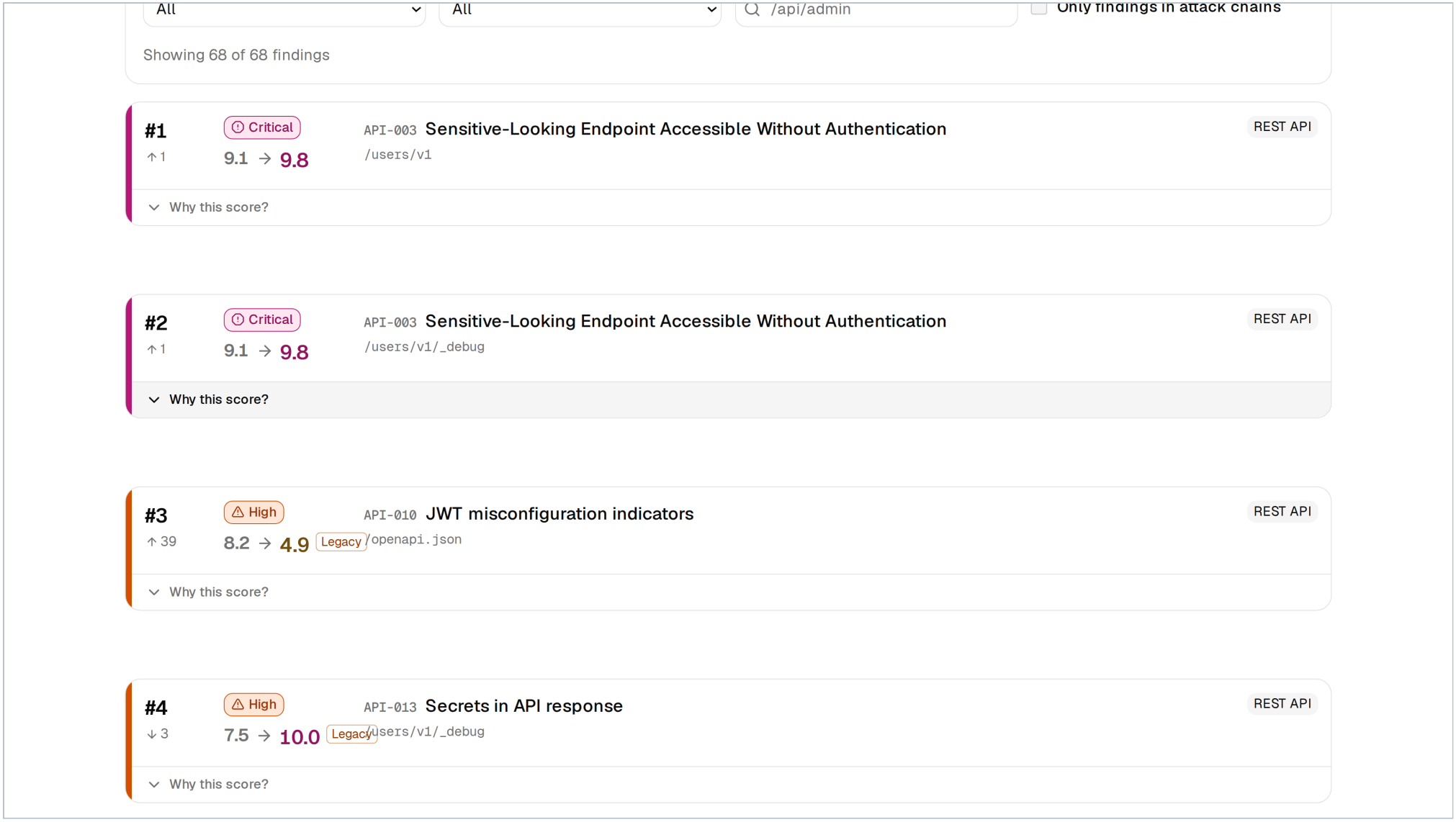


Figure 4. The same findings with "Base score" selected, showing the plain CVSS order; the secrets finding that context ranks first drops to fourth.

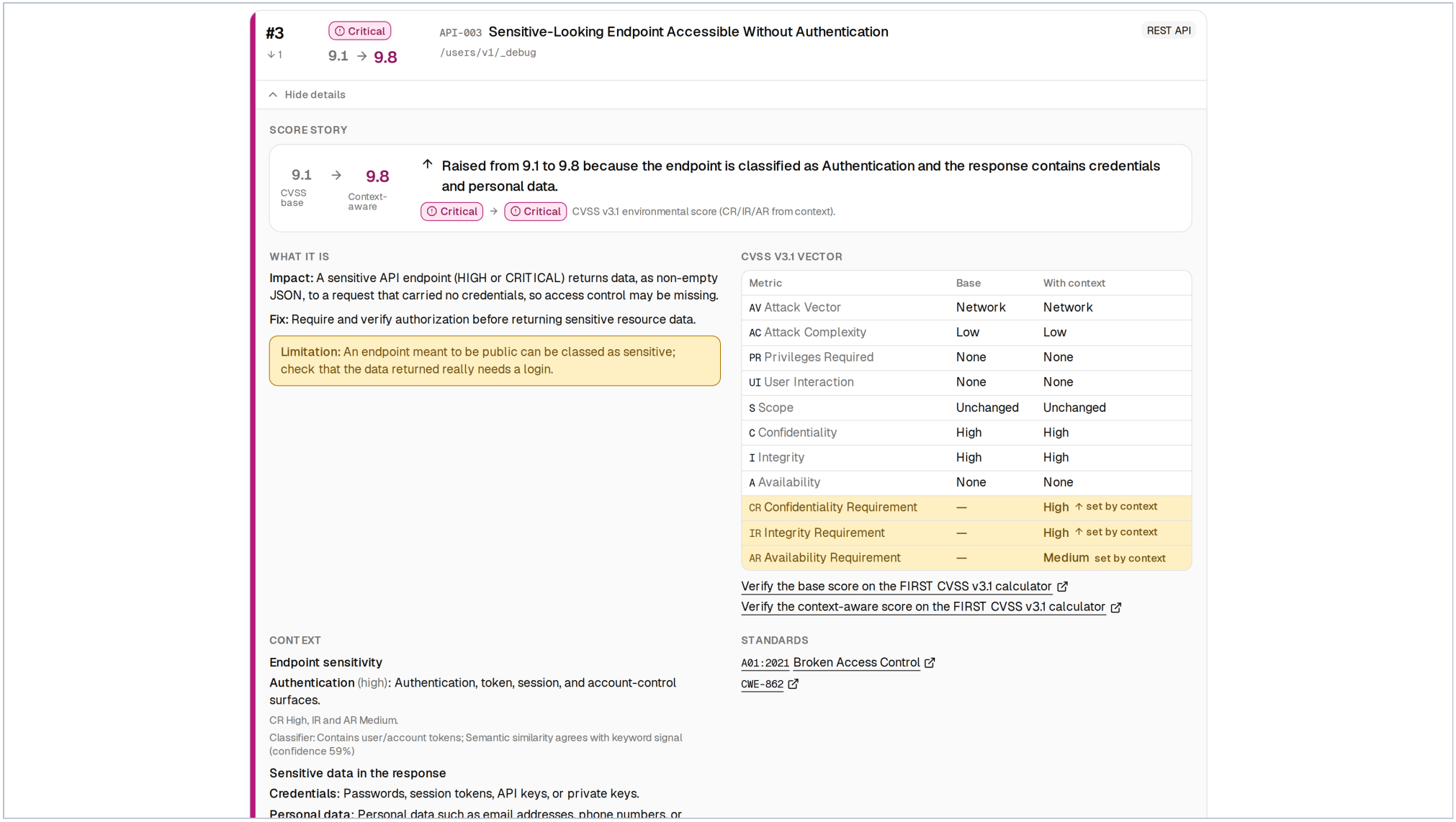
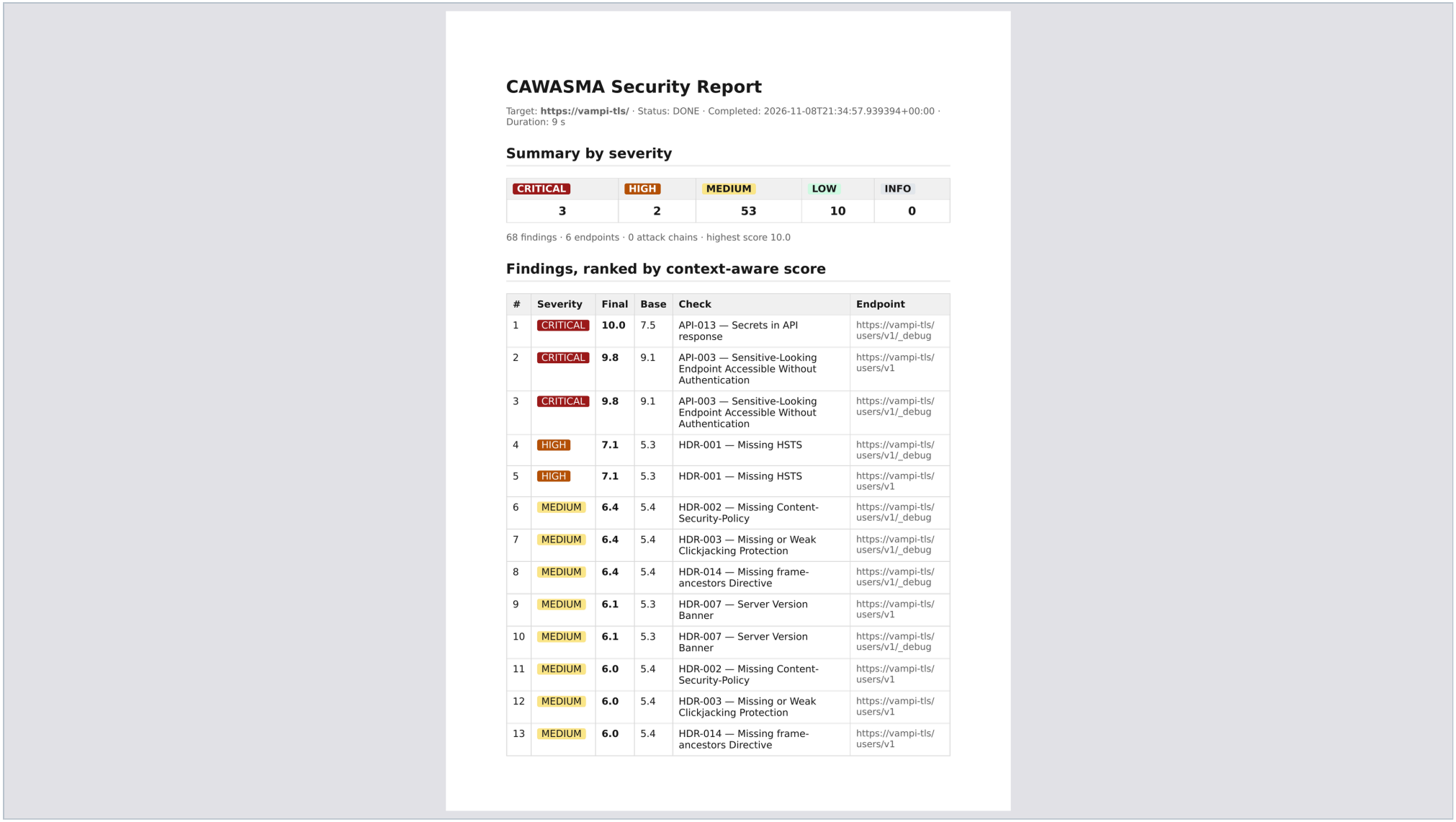


Figure 5. One finding opened: API-003 on `/users/v1/_debug` goes from 9.1 to 9.8, with the CVSS vector, the endpoint class (Authentication, high) and the data found in the response (credentials, personal data).



CAWASMA Security Report

Target: **https://vampi-tls/** · Status: DONE · Completed: 2026-11-08T21:34:57.939394+00:00 · Duration: 9 s

Summary by severity

CRITICAL	HIGH	MEDIUM	LOW	INFO
3	2	53	10	0

68 findings · 6 endpoints · 0 attack chains · highest score 10.0

Findings, ranked by context-aware score

#	Severity	Final	Base	Check	Endpoint
1	CRITICAL	**10.0**	7.5	API-013 — Secrets in API response	https://vampi-tls/users/v1/_debug
2	CRITICAL	**9.8**	9.1	API-003 — Sensitive-Looking Endpoint Accessible Without Authentication	https://vampi-tls/users/v1
3	CRITICAL	**9.8**	9.1	API-003 — Sensitive-Looking Endpoint Accessible Without Authentication	https://vampi-tls/users/v1/_debug
4	HIGH	**7.1**	5.3	HDR-001 — Missing HSTS	https://vampi-tls/users/v1/_debug
5	HIGH	**7.1**	5.3	HDR-001 — Missing HSTS	https://vampi-tls/users/v1
6	MEDIUM	**6.4**	5.4	HDR-002 — Missing Content-Security-Policy	https://vampi-tls/users/v1/_debug
7	MEDIUM	**6.4**	5.4	HDR-003 — Missing or Weak Clickjacking Protection	https://vampi-tls/users/v1/_debug
8	MEDIUM	**6.4**	5.4	HDR-014 — Missing frame-ancestors Directive	https://vampi-tls/users/v1/_debug
9	MEDIUM	**6.1**	5.3	HDR-007 — Server Version Banner	https://vampi-tls/users/v1
10	MEDIUM	**6.1**	5.3	HDR-007 — Server Version Banner	https://vampi-tls/users/v1/_debug
11	MEDIUM	**6.0**	5.4	HDR-002 — Missing Content-Security-Policy	https://vampi-tls/users/v1
12	MEDIUM	**6.0**	5.4	HDR-003 — Missing or Weak Clickjacking Protection	https://vampi-tls/users/v1
13	MEDIUM	**6.0**	5.4	HDR-014 — Missing frame-ancestors Directive	https://vampi-tls/users/v1

Figure 6. First page of the PDF report for the same scan.

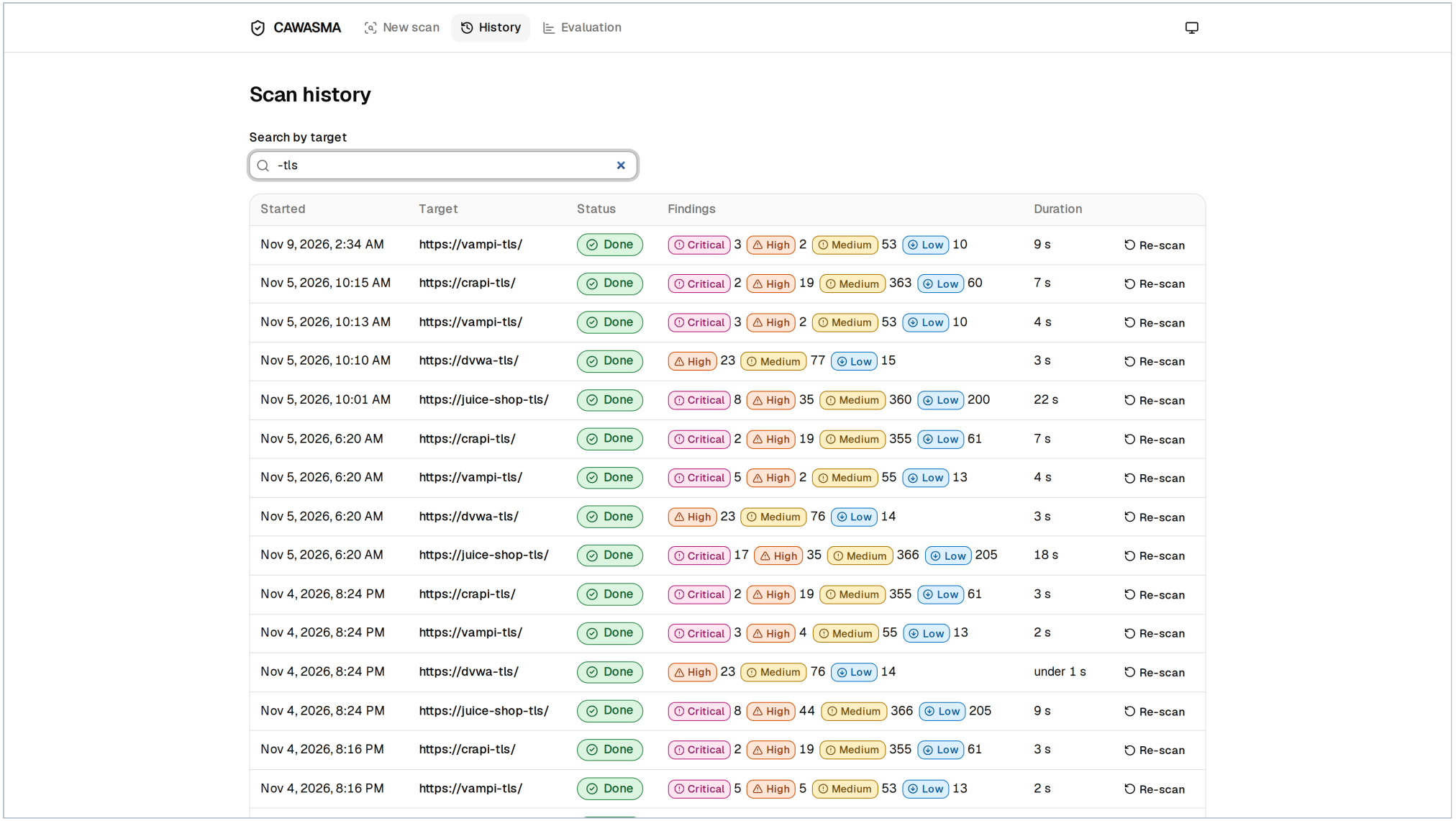


Figure 7. Scan history, filtered with the search box to the local test targets.

4. Plan for what CAWASMA does not cover

CAWASMA currently runs safe, unauthenticated, read-only checks for misconfigurations. It requires the user to confirm authorisation for every scan, scans HTTPS only, sends no credentials, and never requests routes named like state changes (`/logout`, `/reset`, `/create...`). The items below need more than that.

All of them would share two safety rules:

- **Opt-in active mode**, off by default and on top of the existing authorisation confirmation. Passive mode stays exactly as it is.
- **Bounded and non-destructive:** harmless inputs only, no attack payloads, nothing that changes data unless the target is one of our own test targets.

Effort is a rough estimate in working days, including tests and evaluation.

XSS, injection, BOLA/IDOR, authenticated scanning

Item	Why it is out of scope now	What a safe version looks like	Effort	Owner
XSS	Confirming XSS normally means sending a script payload and proving it runs, which is exploitation	**Marker reflection:** send a random inert marker with `"'<>` in a query parameter and report only if it returns unescaped in the HTML, noting the context (text, attribute, script). No script is ever sent. It finds indicators, not proof, and sees only server-side reflection, so DOM-based XSS stays out	2–3	Atta (probe and check); Asim (spec, vector, ground truth)
Injection (SQL, command)	Real tests try to change a query or run a command, which can damage data. Today a single `'"<` probe on API endpoints only flags an error it caused (API-014)	**Error differential:** compare the response to a harmless syntax character against a normal request, and report new database or parser errors as an indicator. No time-based, boolean-extraction or stacked payloads	6–8	Atta (probes, rate limits); Asim (spec, ground truth on DVWA, VAmPI, crAPI)
BOLA / IDOR	Needs a logged-in session for two different users, and a positive test reads someone else's data, which is only acceptable on our own test targets	**Two-account differential:** the user supplies two test accounts; request one account's object with the other's token and compare. Allowed only on targets marked as test targets, read-only (GET), and depends on authenticated scanning	8–10, after authenticated scanning	Asim (method, ground truth on VAmPI, crAPI); Atta (engine)
Authenticated scanning	The engine deliberately keeps no cookies and sends no credentials, so anything behind a login is unseen, and many DVWA and crAPI pages are	**User-supplied session token** (cookie or bearer header), sent only to the in-scope host, never stored or logged, redacted everywhere. CAWASMA would not log in itself. State-changing routes stay blocked, so the session isn't ended by the scan	5–7	Atta (engine, API); Neelam (form field, 1 day); Asim (re-label, re-run evaluation)

The checks marked DROP in `docs/check-triage.md`

The 14 DROP entries are a proposal that Asim and Atta have not yet confirmed; today they still run as legacy checks. For several, the honest plan is to leave them out.

Check	Why it is out of scope now	Safe version, if any	Effort	Owner
API-005 mass assignment	Needs a write request to confirm	On test targets only, with a test account: send one harmless extra field to a throwaway resource and read it back	3–4, after auth	Asim, Atta
API-006 BOLA surface	A numeric id alone proves nothing	Replaced by the BOLA/IDOR item above	included	Asim, Atta
API-008 Deprecation header	Informational, not a weakness	Show as an unscored note	0.5	Atta
API-011 too many fields	Unreliable: a field count says nothing about excess data	Compare response fields with the OpenAPI schema, where one exists	3	Asim, Atta
API-014 injection error	Injection detection rather than misconfiguration	Moves into the injection item above	included	Atta, Asim
AUT-001 Basic auth	Over HTTPS-only targets it is not a misconfiguration by itself	Show as an unscored note	0.5	Atta
AUT-003 short reset token	Needs a reset flow; `/reset` routes are never requested	Test targets only, with a test account and active mode	3, after auth	Asim, Atta
AUT-004 logout keeps session	`/logout` is never requested	With a user-supplied test session: log out, then replay the token. Ends that session, so explicit opt-in	2, after auth	Atta
AUT-005 user enumeration	Needs submitting credentials	Two dummy usernames, a strict request limit, compare the responses; active mode	2	Atta, Asim
AUT-006 default credentials	Trying default passwords is credential testing, a legal and safety line. The current check is unreliable	Never try passwords. At most report the product name as an unscored note	1	Asim
AUT-007 open redirect	Needs an active test	Set the parameter to a reserved domain and read the `Location` header without following it	1.5	Atta
CKI-006 short session cookie	Unreliable: length is not entropy	Sample many sessions and estimate entropy. It creates many sessions, so low value for the cost	2	Leave out
COR-005 missing max-age	Performance, not security	None needed	0	Stays dropped
HDR-012 Expect-CT	The header is deprecated	None needed	0	Stays dropped

Recommendation

Add one item before final submission: the XSS marker-reflection check.

- It's the smallest item (2–3 days) and fits what the scanner already is: a probe already exists, the check needs no login, and it sends nothing harmful.
- It addresses the most visible gap raised in the review with a real, safe feature.
- **Limits:** it reports an indicator, not a proven exploit. No ground truth for XSS exists yet, so Asim would need to add it. The classic DVWA XSS pages sit behind a login, and Juice Shop's best-known XSS is DOM-based, so it may find little on these four targets and needs its own test fixture. Adding it also means re-running the evaluation.

Future work: authenticated scanning first, since BOLA/IDOR, mass assignment and the session checks depend on it; then injection and BOLA/IDOR. Authenticated scanning would add the most coverage, but it touches the engine, the API, the UI and every ground-truth file, so it's too large and too risky to do just before submission.